SEMESTER PROJECT - COMPLETE WALKTHROUGH

Park Ecosystem Portal

Public Park Ecosystem Health Monitoring System & Urban Biodiversity Mapping Portal

Every module, every data source and every formula - and which values are real, simulated or demo.

Park Ecosystem Portal monitors the ecological health of six public parks in Bengaluru. It combines live air-quality and weather observations, biodiversity records published through GBIF, park boundaries and assets mapped in OpenStreetMap, and image analysis with a pre-trained neural network. On top of that data it provides incident triage, maintenance planning, citizen reporting, analytics and a retrieval-based assistant.

This guide explains how to run it, where every number comes from, how each module works and what it cannot do. Each module chapter has the same shape: what the page is for, what is on it, which files and endpoints power it, and the questions an examiner is most likely to ask.

> **Read this first if you have five minutes**
>
> Chapter 5 (Data Sources and Provenance) and Chapter 18 (The AI Pipeline and Its Accuracy) answer the two questions a viva usually opens with: "is the data real?" and "is the AI real?". Chapter 24 lists the limitations and Chapter 25 is a one-page cheat sheet. Stating your own limitations precisely is the strongest thing you can do in a review.

At a glance	
Scope	6 public parks in Bengaluru, 12 modules, plus sign-in, registration and a personal settings page
Frontend	Next.js 13.5 (App Router), React 18, TypeScript, Tailwind, shadcn/ui - 16 page routes
Backend	Node.js + Express 4, Mongoose 8, JWT, TensorFlow.js - 18 routers under /api
Database	MongoDB, 18 collections, GeoJSON with 2dsphere indexes; in-memory for development
Open data	OpenStreetMap, GBIF, GRIIS India, Open-Meteo (forecast + CAMS air quality) - no API keys
Vision	Wildlife recognition: AI vision (Gemini by default, with a free GEMINI_API_KEY) names the species; MobileNetV2 (ImageNet) on the server as evidence and fallback - 3/3 on labelled wildlife photos
Code size	about 35,000 lines across 171 source files (backend src, scripts, tests; frontend app, components, lib)

Contents

1. How to Run It 2
2. Environment Variables 4
3. The Technology Stack 5
4. How the Pieces Fit Together 6
5. Data Sources and Provenance 8
6. Ecosystem Monitoring Dashboard 10
7. GIS & Urban Biodiversity Mapping 12
8. Park Asset Management 13
9. Biodiversity Management 14
10. AI Ecosystem Monitoring 15
11. Environmental Sensor Monitoring 16
12. Citizen Engagement Portal 17
13. Incident & Alert Management 18
14. Maintenance Management 19
15. Analytics & Reports 20
16. AI Environmental Assistant 21
17. Administration 22
18. The AI Pipeline and Its Accuracy 23
19. The Algorithms, in Plain English 25
20. API Overview 27
21. The Database 29
22. Testing 30
23. Deployment 31
24. Limitations 32
25. One-Page Cheat Sheet 33

1. How to Run It

The project is two applications that talk over HTTP: an Express API in backend/ and a Next.js web app in frontend/. A root package.json delegates to both, so every command below runs from the repository root. There is deliberately no combined start script: each server gets its own terminal so its logs can be read and it can be stopped independently.

Requirements

- **Node.js 18.17 or newer** (20 recommended, see .nvmrc) and npm.
- **MongoDB** is optional for development - an in-memory database starts automatically - and required for production (for example the MongoDB Atlas free tier).
- **Internet on first run,** to download the vision model once (about 14 MB). Live Open-Meteo data always needs a connection, as do base-map tiles, species photographs, the GBIF cross-check and address lookup; the rest works offline. No API keys are required.

First-time setup

```
git clone <repository-url>
cd GreenPulse---Escosystem

npm run setup                                  # npm install in backend/ and frontend/
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

The defaults work as they are: a blank MONGODB_URI starts an in-memory MongoDB that is seeded on every boot, and NEXT_PUBLIC_SHOW_DEMO_ACCOUNTS=true lists the demonstration accounts on the login page.

Two terminals

```
# Terminal 1 - API on http://localhost:5000/api
npm run backend:dev
#   wait for: Park Ecosystem Portal API listening on port 5000

# Terminal 2 - web app on http://localhost:3000
npm run frontend:dev
```

On first start the backend seeds the database from the committed open-data snapshot (about ten seconds) and loads the vision model in the background, downloading it once into backend/.cache/mobilenet-v2/ if it is not cached. "Vision model ready" in the log means image analysis is available. To download the model ahead of time, run npm run model:download.

> **For a presentation, consider the production build**
>
> In development mode Next.js compiles each page the first time it is opened, which can take several seconds and looks like a hang. Either open every page once before the review, or build once with "npm run build" and serve with "npm run frontend" instead of "npm run frontend:dev". Also avoid editing code while presenting: the development backend restarts on file changes, and a restart re-seeds the in-memory database.

Signing in

All demonstration accounts use the password greenpulse123. The login page offers one-click buttons for the first four.

Account	Can do
Administrator **admin@greenpulse.gov**	Everything, including users, system settings, the audit log and reseeding.

Account	Can do
Ecologist **ecologist@greenpulse.gov**	Species records, observation verification, AI detection reviews, generating and publishing reports.
Park Officer **officer@greenpulse.gov**	Incidents, work orders, assets, sensor refresh, citizen-report review, row-level exports.
Citizen **citizen@greenpulse.gov**	Report issues, log wildlife sightings, upvote reports, analyse images.

The dashboard, map, biodiversity catalogue, sensors, analytics and assistant are readable without signing in. Public registration always creates a citizen account. The seeder also creates eight further demonstration staff and citizen accounts, twelve in total.

Using a persistent local database

```
# backend/.env
MONGODB_URI=mongodb://127.0.0.1:27017/greenpulse

npm run seed        # REPLACES everything in that database
```

If something goes wrong

- **"Cannot reach the API" on every page -** the backend is not running. Start Terminal 1; the on-screen error says so.
- **"Live data unavailable" in Live Conditions -** you are offline. The virtual sensors then store nothing rather than invent values; everything else works.
- **Image analysis answers 503 -** the model is not cached and cannot be downloaded. Run npm run model:download on a connection; afterwards it works offline.
- **Data looks wrong -** sign in as admin, Administration, System tab, Reseed database (development only). It signs everyone out.
- **Port already in use -** another copy is running. Use it, or stop the other terminal.

2. Environment Variables

Both applications read their configuration from environment files copied from the committed .env.example files. Neither example contains a secret that matters outside development.

backend/.env

Variable	Default	Purpose
PORT	5000	HTTP port. Hosting platforms usually set it.
NODE_ENV	development	production enables the start-up safety checks below.
MONGODB_URI	blank = in-memory	MongoDB connection string. REQUIRED in production.
JWT_SECRET	development placeholder	Token signing secret. REQUIRED in production: at least 32 random characters.
JWT_EXPIRES_IN	7d	Token lifetime.
CORS_ORIGIN	http://localhost:3000	Comma-separated frontend origin(s). REQUIRED in production.
AUTO_SEED	true	Seed on boot when the database has no parks.
SENSOR_SIMULATION_INTERVAL_MS	60000	Sensor refresh interval in milliseconds; 0 disables the background refresh.
LOG_LEVEL	info	debug, info, warn, error or silent.
OPENWEATHER_API_KEY	blank	Optional alternative weather source.
EBIRD_API_KEY	blank	Optional recent bird sightings near a park.

> **Production guards**
>
> With NODE_ENV=production the server refuses to start - and prints what is missing - unless MONGODB_URI is set (the in-memory database is development-only), JWT_SECRET is set, is not the development default and is at least 32 characters, and CORS_ORIGIN is set. POST /api/admin/reseed is refused in production, and error responses stop including internal messages.

frontend/.env.local

Variable	Default	Purpose
NEXT_PUBLIC_API_URL	http://localhost:5000/api in development	Base URL of the API, including /api. Baked in at build time, so it is REQUIRED for a production build.
NEXT_PUBLIC_SHOW_DEMO_ACCOUNTS	false (the example file sets true)	Lists the demo accounts and their password on the login page. When false the credentials are not even in the bundle. Leave it false for a public deployment.

3. The Technology Stack

For each library: what it is, what it does in this project, and why it was chosen. Examiners rarely ask "what is React"; they ask "why did you use this".

Frontend

Technology	What it is, and what it does here
Next.js 13.5 (App Router)	React framework with file-based routing (a folder under app/ becomes a URL) and a production build step. Routing, bundling and code-splitting come built in.
React 18 + TypeScript 5	The UI library, with static types. Every API response shape is declared in lib/types.ts, so a renamed backend field fails compilation instead of showing "undefined".
Tailwind CSS + shadcn/ui (Radix)	Utility-first styling, and accessible primitives (dialogs, sheets, tabs, sliders) whose source lives in components/ui/ so it can be edited.
TanStack Query v5	Server-state cache: loading and error states, background refetching, and invalidation after every write through one query-key factory.
React Hook Form + Zod	Forms and validation. The form schemas mirror the server's Zod schemas, so both sides reject the same input.
Leaflet 1.9 + React-Leaflet 4	The interactive map. Open source, no API key and no usage quota. Base maps: OpenStreetMap, Esri satellite, OpenTopoMap.
Recharts	All charts, written as React components.
jsPDF + autoTable	PDF export of reports and datasets in the browser - no headless browser on the server.
Framer Motion, next-themes, Sonner	Animation, dark and light mode, and toast notifications for the result of every write.

Backend

Technology	What it is, and what it does here
Node.js + Express 4	The runtime and web framework. Middleware runs in order: helmet, compression, CORS, rate limiting, JSON parsing (8 MB, for image uploads), authentication, validation.
MongoDB + Mongoose 8	A document store with schemas, indexes and hooks. GeoJSON is first-class, which is what makes "everything within 1 km" a single indexed query.
mongodb-memory-server	Starts a real MongoDB in memory when MONGODB_URI is blank, so a marker never has to install a database. Development and tests only.
TensorFlow.js (WebAssembly backend)	Runs Google's pre-trained MobileNetV2 inside the Node process - no Python, no GPU, no native build. About 30-80 ms per image on a laptop CPU; falls back to the slower pure-JavaScript backend if WebAssembly cannot start.
jpeg-js, pngjs	Decode uploaded images and re-encode the stored copy (at most 640 px, JPEG).
jsonwebtoken + bcryptjs	Stateless JWT authentication and password hashing. The user is reloaded on every request, so a deactivated account stops working immediately.
Zod	Validates every write route and answers 422 with per-field messages.
helmet, cors, express-rate-limit	Security headers, allowed origins, and throttling: /api 2000 requests per minute per IP (300 in production), sign-in 20 failed attempts per 15 minutes, image analysis 20 per account per minute.
node:test	Node's built-in test runner for the unit and integration suites - no test framework dependency.

4. How the Pieces Fit Together

The request path

Every action in the application follows this path.

```
Browser (React component)
   |  calls a hook, e.g. useDashboard()
lib/hooks/use-api.ts         TanStack Query: cache, loading, errors, invalidation
lib/api/endpoints.ts         one function per API route
lib/api/client.ts            adds the JWT, unwraps the response envelope
   |  HTTP / JSON
backend/src/app.js           helmet, compression, CORS, rate limit, JSON parsing
src/routes/*.routes.js       URL matching (18 routers under /api)
middleware/auth.js           requireAuth (valid token, active user), requireRole
middleware/validate.js       Zod schema from validators/schemas.js
src/controllers/*.js         orchestration; crud.factory.js for the common five
src/services/*.js            the domain logic and every formula
src/models/*.js              Mongoose schemas, indexes, hooks
   |
MongoDB
```

Why the code is split this way

- **Routes only match URLs.** They hold no logic, so the whole API surface can be read in minutes.
- **Controllers orchestrate.** They fetch, call services and shape the response. Eleven modules share list, read, create, update and delete through the CRUD factory.
- **Services hold the thinking.** aqi, biodiversity, ecosystem-score, anomaly, priority, vision, ai-inference, assistant, sensor, report, alert, audit, external. computeEcosystemHealth() has one implementation and many callers, so the number cannot disagree with itself.
- **The frontend never builds a URL.** Components call typed functions; changing a route is a one-line edit.

One response shape everywhere

```
success: { "success": true,  "data": ..., "meta": { pagination or aggregates } }
failure: { "success": false, "error": { "message": ..., "details": { field: reason } } }
```

One error middleware normalises validation errors (422), bad ids (400), duplicates (409), bad tokens (401), unparseable JSON (400) and oversized bodies (413). Anything else is treated as a bug and collapsed to a generic 500.

The role hierarchy

```
citizen (1)  ->  ecologist (2)  ->  officer (3)  ->  admin (4)
```

A route that requires officer also admits an admin. The interface hides what a role cannot use, but that is a courtesy: middleware/auth.js enforces the same rule on the server. The role is read from the database on every request, never trusted from the token payload.

Background jobs

Job	What it does

Job	What it does
Sensor refresh	Every SENSOR_SIMULATION_INTERVAL_MS (default 60 s). Open-Meteo sensors ingest only when the upstream observation time has advanced; simulated sensors emit one reading each while simulation is enabled; park scores are recomputed when anything was ingested.
Overdue sweep	Every 10 minutes, and before every work-order list, calendar and stats read: scheduled orders whose date has passed become overdue.
Vision warm-up	Once at boot: loads (or downloads) the model in the background. A failure is logged and retried on the first analysis.

5. Data Sources and Provenance

Every value is either real open data, a real live observation, a computation over those, or a demonstration record that says so. The interface shows a provenance badge wherever a value appears: Live - Open-Meteo, Simulated, GBIF records, OpenStreetMap, Demo record, Model inference (and Device, for physical sensors, of which none are deployed).

Data	Source	Status
Parks: boundaries, areas, facilities	OpenStreetMap (Overpass API)	Real, committed snapshot
Assets: trees, benches, lamps, paths, water, structures	OpenStreetMap features inside each boundary	Real positions; condition scores and maintenance history are demo values
Species, IUCN category, photographs	GBIF species API	Real, committed snapshot
Species observations	GBIF occurrence records inside each park boundary since Jan 2023, counted per species, park and month	Real - counts are records, not individuals
Invasive / introduced flags	GRIIS India checklist (Darwin Core archive)	Real
Air quality (CPCB AQI), temperature, humidity	Open-Meteo forecast model and CAMS air quality, fetched live	Real "virtual sensors"
Noise, soil moisture, water quality	Generated: AR(1) process with a daily cycle and noise	Simulated, labelled
AI wildlife recognition	AI vision - Gemini by default - with an API key; MobileNetV2 (ImageNet) on the server as evidence and fallback	Real inference, measured accuracy
Accounts, citizen reports, incidents, work orders, derived alerts	Seeder (seeded pseudo-random generator)	Demo records, demo: true
Ecological reports	report.service over all of the above	Computed

The committed snapshot

The open data lives in backend/src/seed/data/open-data/ (parks.json, species.json, observations.json, meta.json and ATTRIBUTION.md), so seeding works offline and gives identical biodiversity figures every time. The current snapshot covers 1 January 2023 to 15 September 2026:

Item	Count in the snapshot
Parks	6 - Cubbon Park, Lalbagh Botanical Gardens, Sankey Tank Park, Jayaprakash Narayan Park, Freedom Park, Coles Park
Species with GBIF records inside the boundaries	587 (the catalogue adds 9 curated regional species with no records, 596 in total)
Observation rows (species x park x month)	4,175
GBIF occurrence records	37,182 - Lalbagh alone holds 33,803; Freedom Park has 1

npm run data:refresh (scripts/fetch-open-data.js) rebuilds the snapshot: Overpass boundaries and features kept by a point-in-polygon test; GBIF occurrence search per month with the simplified boundary as geometry, faceted by species for exact record counts; GBIF species lookups for taxonomy, an English name, the IUCN category and a CC-licensed photo; and the GRIIS India archive for introduced and invasive flags. Requests retry with back-off and

are cached on disk for 24 hours so an interrupted run resumes.

What the seeder adds

Collection	Seeded content
Assets	339 OpenStreetMap features, capped per park (60 trees, 25 benches, 20 lamps, 12 paths, 10 water bodies, 12 structures), each with a demo condition
Sensors	33: AQI, temperature and humidity (Open-Meteo) plus noise and soil (simulated) in every park, and water quality (simulated) in the 3 parks with a mapped water body. Virtual sensors are backfilled with 48 hours of real hourly history.
AI detections	3 real wildlife inferences over openly licensed Wikimedia Commons photos, filed against parks in rotation; none opens an incident
Demo records	12 accounts, 36 citizen reports, 30 incidents, 36 work orders, and the alerts derived from them
Reports	6 generated ecological reports, published

> **GBIF counts are records, not animals**
>
> A GBIF row is one species in one park in one month, and its count is the number of occurrence records - mostly eBird checklists and iNaturalist observations. Ten birders reporting the same kite give ten records. Recording effort differs enormously between parks, and recent months are incomplete because publication lags observation. The interface says "records" wherever this matters.

Live integrations

Service	Key?	Purpose	Cache
Open-Meteo Forecast	No	Current weather, 7-day outlook, hourly history for the virtual sensors	10 min
Open-Meteo Air Quality (CAMS)	No	Hourly pollutant concentrations; the CPCB AQI is computed here	15 min
GBIF	No	Live plausibility cross-check of a species near a park; snapshot build	24 h
OpenStreetMap Nominatim	No	Reverse geocoding of citizen report locations	7 days
TensorFlow Hub	No	MobileNetV2 checkpoint, downloaded once	disk
OpenWeatherMap, eBird	Yes	Optional extras, skipped when no key is set	-

Every call has an 8-second timeout and returns { ok: false, reason } instead of throwing. When Open-Meteo is unreachable the virtual sensors ingest nothing - they go stale and, after 3 hours without data, offline - rather than being filled with invented values. Calls are proxied through the server so one shared cache serves every visitor and keys never reach the browser.

6. Ecosystem Monitoring Dashboard

`/dashboard - Module 1 - public`

What it is for

The landing page. It answers "how are the parks doing right now" on one screen: the composite Ecosystem Health Index and the sub-indices behind it, live weather and air quality, biodiversity, environmental trends, operational queues and a ranking of the parks. A park filter narrows everything to one park. A notice at the top states which data is real, demo or simulated.

Tabs

Tab	What it shows and why it matters
KPI row	Eight tiles: Ecosystem Health, Biodiversity, Air Quality (AQI, badged Live - Open-Meteo), Water Quality, Soil Health, Tree Health, Species Recorded (for example "587 of 596") and Active Alerts. Each is coloured by its normalised score.
Ecosystem Health Index	A gauge and grade, plus Weighted contributions: one bar per sub-index with its weight. Sub-indices without data are listed as "No data" and left out, never counted as 0.
Live Conditions	Current weather and an AQI computed by this project from CAMS concentrations, with the per-pollutant sub-indices, the dominant pollutant, the averaging method and the upstream time in IST. Offline it says "Live data unavailable".
Biodiversity	Records by class, Shannon H', evenness J' and Gini-Simpson 1-D, and the number of species of elevated conservation concern.
Environmental Trends	Daily 0-100 scores over 7, 30 or 90 days. The legend marks water, soil and noise as simulated; a day without readings is a gap, not a zero.
Queues and feeds	Open incidents, reports awaiting review, work orders due (demo records); Active Alerts; Highest Priority incidents; Recent Activity across modules.
Park ranking	Parks Ranked by Ecosystem Health, with biodiversity alongside, and a sensor-network strip (online, warning, offline).

What powers it

Layer	Detail
Page	frontend/app/(app)/dashboard/page.tsx, components/shared/conditions-panel.tsx
API	GET /api/dashboard/overview, /dashboard/trend, /dashboard/activity; GET /api/integrations/park-conditions
Backend	controllers/dashboard.controller.js - the overview returns the whole page in one round trip
Services	ecosystem-score.service.js, biodiversity.service.js, aqi.service.js, external.service.js

Likely viva questions

- **How is the health score calculated?** EHI = sum(w_k * S_k) / sum(w_k) over the sub-indices that have data, with air 0.25, water 0.20, tree 0.20, biodiversity 0.20 and soil 0.15. Dividing by the weights actually present means a park without a water sensor is scored on what it has, not punished with a zero.
- **Why does Freedom Park score so low?** Its biodiversity sub-index is almost zero because GBIF has a single record inside its boundary - missing survey effort, not a dead park - and it has no mapped trees or water, so those are "No data".
- **Why can the Air Quality KPI differ from Live Conditions?** The KPI is the mean of the parks' stored virtual-sensor readings; the panel is a fresh query for the selected location (the mean park position when no park

is selected).

7. GIS & Urban Biodiversity Mapping

`/map - Module 2 - public - sidebar: Biodiversity Map`

What it is for

An interactive Leaflet map with eight switchable layers. The module has no collection of its own: every layer is a live projection of data another module owns - a tree pin is the asset register's record of that tree - so the map cannot go stale relative to the rest of the system.

Tabs

Tab	What it shows and why it matters
Layers	Parks (OSM boundaries), Trees & Plants, Water Bodies, Walking Trails (OSM), Wildlife Records (the latest 500 verified GBIF-based observations), Pollution Hotspots (demo incidents), Sensors, Citizen Reports. Each chip shows its feature count.
Search	Matches across visible layers. Non-matching features are dimmed, not hidden, to keep the spatial context. A hit flies the map to the feature.
Selected panel	The full record of the clicked feature with its provenance badge; asset condition is labelled "demonstration value".
Base maps	OpenStreetMap, Esri satellite or OpenTopoMap through Leaflet's own control. The tiles need a connection; the data layers do not.

What powers it

Layer	Detail
Page	frontend/app/(app)/map/page.tsx, components/map/biodiversity-map.tsx, components/map/layer-config.ts
API	GET /api/gis/layers, /gis/heatmap, /gis/within
Backend	controllers/gis.controller.js - composes GeoJSON FeatureCollections from other collections
Data	GeoJSON with 2dsphere indexes; pollution circles sized by the incident's computed priority

Likely viva questions

- **Why GeoJSON?** Leaflet consumes it almost directly, and MongoDB's 2dsphere index understands it, so "what is within 1 km" (/gis/within, /parks/near) is one indexed $near query.
- **What is the coordinate trap?** GeoJSON is [longitude, latitude]; Leaflet is [latitude, longitude]. Backwards, a Bengaluru park lands in the Indian Ocean. Conversion happens only in frontend/lib/api/geo.ts, and the API rejects [0, 0] as "no location picked".

8. Park Asset Management

`/assets - Module 3 - public to read; officer to edit - sidebar: Park Assets`

What it is for

The inventory of physical assets - trees, plants, benches, lakes, paths, lights and structures. Positions and types are real OpenStreetMap features; condition scores and maintenance histories are demonstration values, badged "Demo condition". Tree and plant condition feeds the tree-health sub-index.

Tabs

Tab	What it shows and why it matters
Summary tiles	Total assets, Mean condition, Needs attention (condition below 50) and Maintenance spend.
Condition by Asset Type	Count and mean condition per type, coloured by condition.
Inventory table	Filter by type, status and park; search by name, code or notes. Each row carries an OpenStreetMap badge and, where applicable, a Demo condition badge.
History / Log maintenance	The history drawer lists maintenance records and related work orders. Log maintenance records the work and raises the condition by 10 (the API also accepts an explicit condition after the work).
Add, Edit, Retire	Officers add and edit; retiring is a soft delete (admin), so history and work orders stay intact.

What powers it

Layer	Detail
Page	frontend/app/(app)/assets/page.tsx
API	GET/POST/PATCH/DELETE /api/assets; GET /assets/stats, /assets/:id/history; POST /assets/:id/maintenance
Backend	controllers/asset.controller.js on the CRUD factory; models/Asset.js
Notable	status is derived from condition in a pre-save hook, so the two can never disagree; codes (TRE-0001 ...) come from an atomic counter

Likely viva questions

- **Why one collection for seven types?** They share almost every field. Type-specific data goes in a free-form attributes map, so queries, the table and the map layer stay generic.
- **Is the condition data real?** No - nobody has surveyed these assets. The positions, species and types are real OpenStreetMap data; condition and history are demo values and are labelled so.

9. Biodiversity Management

`/biodiversity - Module 4 - public; ecologist to curate and verify`

What it is for

The species catalogue and the diversity indices computed over GBIF occurrence records inside each park boundary since January 2023. Only verified observations count; GBIF rows are imported as verified because they are already published records.

Tabs

Tab	What it shows and why it matters
Indices	Biodiversity Score, species richness, Shannon-Wiener and Pielou evenness; How the score is derived (five formula cards with this data's values and the composite); Composition; Threatened; Invasive records (GRIIS India); Per-Taxocene Indices; Abundance Distribution; Seasonality; and the Index Calculator.
Catalogue	Species cards with IUCN badges - "Not Evaluated" kept distinct from "Least Concern" - and "Invasive in India (GRIIS)" or "Introduced" badges. The species sheet links to GBIF.org and offers Cross-check against GBIF.
Observations	Records filterable by verification and source. GBIF rows read "N GBIF records" and show a month, not a day. Ecologists can Verify or Withdraw.
Compare	Biodiversity by Park: every park on the same indices.

What powers it

Layer	Detail
Page	frontend/app/(app)/biodiversity/page.tsx
API	GET /api/biodiversity/indices, /compare, /seasonality, /species, /observations; POST /indices/preview, /observations/:id/verify
Service	services/biodiversity.service.js - all index mathematics
Models	Species (catalogue only) and Observation (the abundance records)
External	GET /api/integrations/gbif/:speciesId - live cross-check verdict

Likely viva questions

- **What does Shannon measure?** H' = -sum(p_i * ln p_i): the uncertainty in guessing the species of a randomly drawn record. It rises with both richness and evenness; one species gives 0.
- **Show me the maths is real.** Index Calculator: "100, 1, 1, 1" gives H' = 0.16, J' = 0.12; "25, 25, 25, 25" gives H' = 1.39, J' = 1.00. Same richness, different ecological health.
- **Why does Sankey Tank outscore Lalbagh?**A sampling artefact. With 96 records most species appear once or twice, so evenness approaches 1; Lalbagh has 33,803. The indices describe what has been recorded, not how many animals live there.
- **Is pooling birds and plants valid?** Not really - ecologists compute diversity within a taxocene. The pooled score is reported because a manager needs one number, and the per-taxocene table is the rigorous comparison.

10. AI Ecosystem Monitoring

`/ai - Module 5 - public to read; sign-in to analyse - sidebar: AI Monitoring`

What it is for

Wildlife recognition from photos - species named, every finding reviewed by a park officer. With a language-model key set (GEMINI_API_KEY is free), AI vision names the animal species (common and scientific); a pre-trained MobileNetV2 always runs as evidence and as the fallback engine. Chapter 18 has the full pipeline.

Tabs

Tab	What it shows and why it matters
Analyse	The task is fixed to Wildlife recognition. Upload a JPEG or PNG (up to 6 MB) or give an image URL, optionally choose a park, then Analyse image. The result leads with the species name (scientific name below, wildlife label as a chip) and the Recommended action; then confidence, Label probabilities, Caveats, Evidence, Top ImageNet classes, the engine and timings.
Detections	The latest 24 detections, each with its species and engine. Park officers mark each Correct, or Wrong with the label it should have had.
Model	Whether AI vision is active (and which provider), the MobileNetV2 model card (weights, input size, runtime and backend, measured accuracy) and the wildlife labels with their severities and "N/M correct".
Performance	Total detections, Awaiting review, Confirmed, Observed precision (confirmed / reviewed), detections and mean confidence, confidence distribution, findings by severity.

What powers it

Layer	Detail
Page	frontend/app/(app)/ai/page.tsx
API	POST /api/ai/analyze, /ai/:id/review; GET /ai/tasks, /ai/stats, /ai/gallery, /ai/detections, /ai/images/:id
Services	vision.service.js (fetch, decode, network), ai-inference.service.js (wildlife label mapping, engine choice), llm-vision.service.js (species identification) and llm-provider.service.js (Gemini, Groq, OpenAI-compatible or Claude)
Models	AiDetection (engine, species, probabilities, top-5 ImageNet classes, evidence, review), AiImage (stored JPEG, unique SHA-256)

Likely viva questions

- **Is the AI real?** Yes - genuine inference on the submitted pixels. AI vision (Gemini by default) identifies the animal species under a strict JSON schema; MobileNetV2 (Google's ImageNet checkpoint, not trained by this project) is the evidence and the fallback.
- **How accurate is it?** Fallback engine: 3 of 3 labelled wildlife photos (pond heron and bee-eater as Bird, plain tiger as Butterfly or moth). Three photos is a smoke test, not an error rate.
- **Why is a pond heron a "bittern" without a key?** The fallback names species at ImageNet granularity; most Indian species are not in ImageNet. AI vision names it Indian pond heron, Ardeola grayii.

11. Environmental Sensor Monitoring

`/sensors - Module 6 - public; officer to refresh`

What it is for

The sensor network, normalised onto a common 0-100 scale and screened by a three-detector anomaly ensemble. There is no physical deployment: AQI, temperature and humidity sensors are virtual Open-Meteo observations for each park's coordinates; noise, soil moisture and water quality are simulated, to exercise the same ingestion, anomaly and alert path a physical gateway would use. The page says exactly this at the top.

Tabs

Tab	What it shows and why it matters
Network tiles	Deployed (with the split by source, for example "18 Open-Meteo - 15 simulated"), Online, Warning, Offline (excluded from scoring) and Maintenance.
Normalised Scores by Sensor Type	Mean score per type with its source badges and a note on how the raw unit is mapped.
Device grid	One card per sensor: value, normalised score, status, source badge, threshold breach and staleness, filterable by type. Officers get Refresh readings.
Detail sheet	Reading history over 6 h to 7 d with warning thresholds as dashed lines and flagged anomalies as red points; min, max, mean, median, sigma; the anomaly list with z-scores and reasons; calibration.

What powers it

Layer	Detail
Page	frontend/app/(app)/sensors/page.tsx
API	GET /api/sensors/live, /:id/readings, /:id/anomalies; POST /sensors/refresh (officer), /:id/readings (device sensors only)
Services	sensor.service.js (refresh, simulation, ingestion), anomaly.service.js (detection), external.service.js (Open-Meteo)
Models	Sensor (source: open-meteo | simulated | device, thresholds, cached value) and SensorReading

Likely viva questions

- **Are the sensors real?** Half of them are real data without real hardware: every stored AQI, temperature and humidity reading is an Open-Meteo observation, ingested only when the upstream time advances. Noise, soil and water are simulated and labelled.
- **Why did Refresh readings add 0 observations?** Because Open-Meteo had no newer observation. The system refuses to duplicate or invent readings.
- **Why are two parks' AQI identical?** CAMS and the forecast model are gridded; one cell can cover several nearby parks.
- **How would real hardware plug in?** Register a sensor with source "device"; a gateway POSTs to /api/sensors/:id/readings and nothing downstream changes.

12. Citizen Engagement Portal

`/citizen - Module 7 - public to read; sign-in to report and upvote; officer to review`

What it is for

Where the public reports issues, logs wildlife sightings and gives feedback or suggestions. An officer reviews every submission: an accepted issue becomes a tracked incident, an accepted sighting becomes a verified observation that enters the biodiversity indices. The seeded reports are demo records.

Tabs

Tab	What it shows and why it matters
All reports	Filter by category, status and park. Each card shows its CR- reference, status, upvote button and, once accepted, "Became incident INC-... - status". Officers and admins see Review on submitted and in-review reports.
My contributions	The signed-in user's own reports, statuses and upvotes received.
Community impact	Submissions by category, participation by park, acceptance rate and top contributors.
Submit a report	Issue, Wildlife sighting, Feedback or Suggestion; title, description, park, species (sightings); location from Park centre or Use my location, with a reverse-geocoded address.
Review report	Accept, Hold or Reject. Accepting an issue asks for incident type, severity and people affected (they feed the triage score); accepting a sighting needs a species and a count.

What powers it

Layer	Detail
Page	frontend/app/(app)/citizen/page.tsx
API	GET /api/citizen/reports, /my-reports, /my-upvotes, /stats; POST /reports, /reports/:id/upvote, /reports/:id/review; DELETE /reports/:id/upvote
Backend	controllers/citizen.controller.js
Model	CitizenReport (upvotedBy, linkedIncident, species)

Likely viva questions

- **Can one person inflate upvotes?** No. The upvote is one conditional atomic update - add this user and increment only if the user is not already in upvotedBy - so repeats count once. The voter list is never sent to clients.
- **Do upvotes change priority?** Yes, slightly: the count is mirrored onto the linked incident and feeds its community term, log-scaled and weighted 0.10, so popularity cannot outrank a hazard.
- **Why keep reports separate from incidents?** Reports are unverified public input; incidents are work items. Acceptance is the gate, and the link preserves the trail from citizen to resolution.

13. Incident & Alert Management

`/incidents - Module 8 - officer (the public alert feed is shown to others)`

What it is for

The operational core. Incidents are ordered by a score computed from their own attributes, never typed in. Incident records carry reporter identities and exact hazard locations, so every incident route - including reads - requires an officer. The seeded incidents are demo records.

Tabs

Tab	What it shows and why it matters
Triage queue	Open incidents highest score first, each with its live score, priority, status, overdue flag and five factor values; Open, Overdue and Critical tiles.
All incidents	The full history with filters, including resolved and closed incidents.
Alerts	The deduplicated alert stream from sensors and incidents: Acknowledge, Resolve, or acknowledge all.
Statistics	Incidents by Type, By Priority Band, and Response Time Against Target.
Detail sheet	Score breakdown and timeline. Actions: Assign to a member of staff, Raise a work order, Update status & details (severity and people affected re-score it), Resolution notes and Mark resolved. Report an incident creates one by hand.

What powers it

Layer	Detail
Page	frontend/app/(app)/incidents/page.tsx
API	GET /api/incidents, /incidents/triage, /incidents/stats; POST /incidents, /:id/assign, /:id/resolve, /:id/work-order; GET/POST /api/alerts
Services	priority.service.js (triage), alert.service.js (deduplicated alerts)
Notable	priority and priorityScore are recomputed on every write and ignored if sent; one incident cannot have two open work orders (409)

Likely viva questions

- **How is priority calculated?** P = 100 * (0.35*H + 0.25*S + 0.20*E + 0.10*U + 0.10*C): hazard of the type (fire 1.0, vandalism 0.4), severity, log-scaled exposure, urgency from age, log-scaled community upvotes.
- **Explain the ageing term.** U(t) = 1 - e^(-t/tau), tau being the type's response target. It climbs quickly and saturates: at most 10 points, so a week-old vandalism report can never outrank a new fire.
- **How do you avoid alert spam?** Alerts are keyed by condition (dedupeKey). A repeat increments an occurrence counter instead of adding a row, and the alert auto-resolves when the condition clears.

14. Maintenance Management

`/maintenance - Module 9 - officer`

What it is for

Work orders - the planned counterpart to incidents. Completing one logs the work against its asset and restores the asset's condition. The seeded work orders are demo records.

Tabs

Tab	What it shows and why it matters
Work orders	Filterable list with progress, priority, assignee and overdue flags. New work order schedules one, optionally against an asset and a member of staff.
Calendar	A month grid of orders on their scheduled days, navigable month to month.
Workload	Estimated Cost by Work Type, Orders by Status, and Open Assignments per Person.
Detail sheet	Update progress with a slider and Save progress. 100 % completes the order, appends a maintenance record to the linked asset and raises its condition by 15. Reschedule, Cancel work order, Delete.

What powers it

Layer	Detail
Page	frontend/app/(app)/maintenance/page.tsx
API	GET /api/maintenance, /maintenance/calendar, /maintenance/stats; POST /maintenance; PATCH /:id, /:id/progress
Backend	controllers/maintenance.controller.js; models/WorkOrder.js keeps progress, status and completion time consistent

Likely viva questions

- **Why separate from incidents?** Different lifecycles: an incident is reported, triaged and resolved; a work order is scheduled, worked and completed. Most work orders come from a schedule, not an incident.
- **How does an order become overdue?** It is derived: a sweep every 10 minutes, and before every list, calendar and stats read, marks scheduled orders past their date as overdue.

15. Analytics & Reports

`/analytics - Module 10 - public aggregates; ecologist for reports; officer for exports`

What it is for

Cross-module trend analysis, park comparison, ecological reports computed from the data, and exports. Aggregation happens in MongoDB pipelines; the page labels which series are real and which are demo or simulated.

Tabs

Tab	What it shows and why it matters
Overview	Health Sub-Indices, Operational Summary and Threatened Species Recorded for the selected window and park.
Trends	Environmental Indicators with anomaly counts, Biodiversity Over Time (Shannon recomputed per month), Incident Volume & Backlog, Citizen Participation.
Compare	Ecosystem Health by Park on all sub-indices, with a comparison table.
Reports	Generate report (type, park, period) creates a draft whose metrics, findings and recommendations are computed from recorded data; Publish makes it visible to everyone; Archive; Export PDF. Drafts are visible to ecologists and above only.
Export	Incidents, assets, observations, citizen reports and work orders as CSV or PDF - officers and administrators only.

What powers it

Layer	Detail
Page	frontend/app/(app)/analytics/page.tsx
API	GET /api/analytics/summary, /environmental-trend, /biodiversity-trend, /incident-trend, /engagement, /park-comparison, /export, /reports; POST /reports/generate; PATCH /reports/:id
Services	report.service.js (generated reports), plus the scoring services
Model	EcoReport - a frozen metric snapshot, findings, recommendations, draft/published/archived

Likely viva questions

- **How is a report "computed"?** Each sentence comes from a stated rule over the frozen metrics - weakest sub-index below 55, evenness below 0.6, invasive records present, mean AQI above 100, overdue work orders and so on. A metric with no data produces a sentence saying so.
- **Why is the biodiversity trend special?** Shannon is recomputed from each month's own records. Averaging an index across months would be meaningless: it is a property of a distribution.

16. AI Environmental Assistant

`/assistant - Module 11 - public - sidebar: Eco Assistant`

What it is for

Question answering over the live database. Retrieval is genuine - TF-IDF weighting and cosine similarity over a corpus rebuilt from MongoDB - and answers are composed from templates over the retrieved records and live computations. It is not a large language model, and it says so.

Tabs

Tab	What it shows and why it matters
Conversation	Ask in plain language; each answer shows its intent, confidence and response time. New conversation starts a fresh thread; a reload restores the last one.
Try asking	Suggested questions, for example "How is the air quality at Cubbon Park?" and "How diverse are the species recorded at Lalbagh?".
Sources	The records the retrieval step surfaced, with similarity scores and links to their modules.
How it works	The retrieval mathematics, stated on the page.

What powers it

Layer	Detail
Page	frontend/app/(app)/assistant/page.tsx
API	POST /api/assistant/ask, /assistant/search; GET /assistant/suggestions, /assistant/history/:sessionId
Service	services/assistant.service.js - tokenising, intent routing, TF-IDF index, answer templates
Model	ChatMessage - both turns with citations; a signed-in conversation is private to its account

Likely viva questions

- **Is it an LLM?** No. Retrieval is real; generation is templates. Every figure is read from the database and traceable, so it cannot invent a number.
- **Why cosine, not a dot product?** Document lengths vary by an order of magnitude; without length normalisation long species descriptions would win every query.
- **Why does naming a park not hijack the intent?** A park name sets the scope; if a more specific intent also scored, that intent is used - so "air quality at Cubbon Park" is answered about air.

17. Administration

`/admin - Module 12 - admin only`

What it is for

User management, system configuration, the audit trail and maintenance actions. The consequential control is the health-index weight editor.

Tabs

Tab	What it shows and why it matters
Users	Add user, edit role and home park, deactivate. Elevated roles can only be granted here; an admin cannot change their own role or deactivate themselves.
Settings	Ecosystem Health Index Weights (must sum to 1.00, with Normalise to 1.00; saving re-scores every park); Algorithm Parameters & Switches (anomaly z-score threshold, AI auto-escalation confidence floor, sensor simulation, public reporting); Organisation.
Audit log	Append-only record of privileged writes with before-and-after fields.
System	Runtime and collection sizes; Public API Integrations with Reachable badges and Clear integration cache; Maintenance Actions: Recompute, Reindex, Reseed database (development only).

What powers it

Layer	Detail
Page	frontend/app/(app)/admin/page.tsx
API	/api/admin/users, /admin/settings, /admin/audit-log, /admin/stats, /admin/recompute-scores, /admin/reindex-assistant, /admin/reseed; /api/integrations/status
Backend	controllers/admin.controller.js - the whole router is behind requireRole("admin")
Models	User, Setting (singleton), AuditLog

Likely viva questions

- **Why validate that weights sum to 1?** They define the composite. Weights summing to 1.3 would silently inflate every score; and re-scoring every park on save avoids parks being scored under two formulas.
- **What stops self-promotion?** Registration hard-codes the citizen role and ignores a role in the body; the role is re-read from the database on every request.

18. The AI Pipeline and Its Accuracy

The model is Google's MobileNetV2 1.0 / 224 ImageNet classification checkpoint from TensorFlow Hub (imagenet/mobilenet_v2_100_224/classification/2), run by TensorFlow.js on its WebAssembly backend. It is downloaded once (about 14 MB) into backend/.cache/mobilenet-v2/ - at build time on Render - and loaded from disk after that. Nothing was trained or fine-tuned by this project. AI Monitoring runs one task, Wildlife recognition. With a language-model key set (GEMINI_API_KEY is free; Groq, a local model or Claude also work), AI vision (VISION_MODEL, default gemini-3.5-flash-lite on Gemini) identifies the animal species; MobileNetV2 is then the evidence shown with the result, and the fallback if the API fails.

What happens on POST /api/ai/analyze

```
requireAuth, rate limit 20 analyses / account / minute
loadImageBytes   URL: DNS check refuses loopback, private, link-local, CGNAT,
                 multicast; redirects followed manually (max 3), each re-checked;
                 15 s timeout, 8 MB max.   data URL: base64 decode, 8 MB max
decode           JPEG or PNG only, detected from the bytes, at least 16x16
MobileNetV2      bilinear resize to 224x224, scale to [0,1], 1001 logits,
                 softmax, drop "background" -> 1000 ImageNet probabilities
interpret(task)  pool ImageNet animal class groups -> wildlife label scores
llm-vision       with a key: stored JPEG -> Gemini (or another provider),
                 strict JSON schema in the wildlife labels -> species, action
storeImage       <= 640 px JPEG, stored once per SHA-256 in AiImage
recordDetection  engine, species, probabilities, evidence, notes
                 -> review queue (nothing is dispatched automatically)
```

Turning ImageNet into wildlife labels

ImageNet has no park-management categories, so the fallback builds the wildlife labels from signals the network does produce. A group's evidence is the probability mass on its members, E_g = sum of p_i over the group - the 59 bird classes become one "bird" signal. Scores are normalised into a probability vector, P(k) = s_k / sum(s_j); the prediction is the argmax and the confidence is 100 * max P.

Task	Labels	How it is scored
Wildlife recognition	Bird / Butterfly or moth / Other insect or invertebrate / Reptile or amphibian / Wild mammal / Domestic or stray animal / No animal	ImageNet animal groups; the closest ImageNet class is the fallback name, AI vision names the species.

Measured accuracy (npm run eval:vision)

scripts/evaluate-vision.js runs the 3 openly licensed Wikimedia Commons wildlife photographs listed in src/seed/data/sample-images/attribution.json through the real pipeline and counts a prediction correct when it is one of that photo's accepted labels.

Task	Correct	Misses
Wildlife recognition	3 / 3	Correct at group level: the Indian pond heron is "Bird", closest ImageNet class "bittern"; the plain tiger is "Butterfly or moth".

> **The sample-size caveat - say it before you are asked**
>
> Three photographs is a smoke test, not a validated error rate. A real figure needs a larger held-out labelled set, which the review queue collects. npm run eval:vision -- --llm measures AI vision on the same photos.

Review

- **Nothing is dispatched automatically.** Every detection, from either engine, enters the review queue.
- **Review builds ground truth.** A park officer confirms a detection or rejects it with a corrected label from the task's own vocabulary. Observed precision = 100 * confirmed / (confirmed + rejected), shown once a review exists.
- **Demonstration photo:** wildlife-indian-pond-heron.jpg gives "Bird" - "Bittern" without a key, "Indian pond heron" (Ardeola grayii) with one.

19. The Algorithms, in Plain English

A summary of every formula. docs/algorithms.md has the derivations, constants and worked examples, reproduced by calling the service functions themselves.

Air Quality Index (aqi.service.js)

```
Sub-index, by linear interpolation inside the CPCB band containing C:
  I = ((I_high - I_low) / (C_high - C_low)) * (C - C_low) + I_low

Averaging:  24-hour means for PM2.5, PM10, NO2, SO2; 8-hour means for CO, O3
            a pollutant counts only with >= 75 % of its hours present
Truncation: C is truncated to the table precision first (whole ug/m3; CO 0.1)
Units:      Open-Meteo CO is ug/m3; divided by 1000 for the mg/m3 CO table

AQI = max(sub-indices)          no qualifying pollutant -> AQI is null
```

The MAXIMUM, not the mean: air is only as clean as its worst pollutant. Truncation matters because the printed bands have gaps - PM2.5 [0, 30] then [31, 60] - and 30.9 would otherwise fall through to 500. For scoring, AQI is inverted band by band: AQI 0-50 scores 100, 100 scores 80, 200 scores 60, 300 scores 40, 400 scores 20, 500 scores 0.

Ecosystem Health Index (ecosystem-score.service.js)

```
EHI = sum(w_k * S_k) / sum(w_k)     over k with S_k not null

air 0.25 (AQI score and noise score)   water 0.20   tree 0.20
biodiversity 0.20                      soil 0.15

grades: >= 85 excellent, >= 70 good, >= 55 moderate, >= 40 poor, else critical
```

Missing is null, never 0. A sensor counts only if it is active, not offline and has reported. Readings are normalised first: noise is lower-is-better (35 dB scores 100, 85 dB scores 0); water and soil are already 0-100; temperature and humidity use an optimal band (centre 24 C, half-width 14; centre 55 %, half-width 35) but do not enter the EHI. Weights live in Setting and are editable by an admin.

Biodiversity (biodiversity.service.js)

```
Richness        S = number of distinct species
Shannon         H' = -sum(p_i * ln p_i)          p_i = n_i / N
Pielou          J' = H' / ln(S)                  0 when S = 1
Simpson         D = sum(p_i^2), reported as 1 - D
Margalef        D_Mg = (S - 1) / ln(N)
Berger-Parker   d = max(p_i)

Score = 100 * (0.35*H_n + 0.25*J' + 0.20*R_n + 0.20*C)
  H_n = min(1, H' / ln 40)   R_n = min(1, S / 40)
  C   = conservation term: IUCN weights NT 2, VU 3, EN 4, CR 5, others 1
```

Only verified observations count, and n_i is a number of GBIF records. "Not Evaluated" and "Least Concern" both weigh 1, so neither inflates the score. Per-class indices (byClassIndices) are returned alongside the pooled figure.

Incident triage (priority.service.js)

```
P = 100 * (0.35*H + 0.25*S + 0.20*E + 0.10*U + 0.10*C)

  H  hazard of the type: fire 1.00, water pollution 0.80, air pollution 0.75,
     tree fall 0.70, infrastructure 0.60, dumping 0.50, dead animal 0.45,
     vandalism 0.40
  S  severity (s - 1) / 4
  E  ln(1 + people) / ln(5001), capped at 1
  U  1 - e^(-t / tau), tau = response target (fire 0.5 h ... vandalism 48 h)
  C  ln(1 + upvotes) / ln(201), capped at 1

bands: >= 75 critical, >= 55 high, >= 35 medium, else low
```

Anomaly detection (anomaly.service.js)

```
Z-score           z = (x - mean) / sd                flag |z| > 3 (configurable)
Modified z-score  M = 0.6745 * (x - median) / MAD    flag |M| > 3.5
Tukey IQR fence   outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR], only when IQR > 0

Window: previous 50 readings; suppressed below 8.  Flag when >= 2 of 3 agree.
```

0.6745 is the 75th percentile of the standard normal, which makes MAD a consistent estimator of the standard deviation, so 3.5 is comparable to 3. The median has a 50 % breakdown point, so the robust detector is not fooled by the outliers it is looking for. An anomaly raises an alert: high if all three voted, else medium.

Simulated sensors (sensor.service.js)

```
x_t = 0.7 * x_(t-1) + 0.3 * (base + A * sin(2*pi*(h - phase) / 24)) + sigma * z

h = local hour in Asia/Kolkata;  z ~ N(0,1) by Box-Muller
with probability 0.03 a spike of A * (2.5 + 2u) is injected
```

Only noise, soil moisture and water quality are generated this way. Note that sin(2*pi*(h - phase)/24) peaks six hours after "phase" - the code calls it peakHour, which is a naming slip worth admitting if asked.

Retrieval (assistant.service.js)

```
tf(t,d)  = count of t in d / length of d
idf(t)   = ln(N / (1 + n_t)) + 1
cos(q,d) = dot(q, d) / (||q|| * ||d||)         top 5 positive matches cited
```

Integrity rules

- **Reference codes** (INC-YYYY-NNNN, WO-, CR-, TRE-) come from a Counter document incremented atomically with $inc and initialised from the highest stored code, so deleting a record can never make the next code collide.
- **Upvotes** are a single conditional update, so a double-click or replay counts once; the linked incident is re-scored after each change.

20. API Overview

Base URL http://localhost:5000/api. GET /api lists every module mount point and GET /api/health reports database status (503 when disconnected). docs/api-reference.md documents every route, role and body.

Conventions

- **Authentication:** Authorization: Bearer <jwt> from POST /auth/login or /auth/register; tokens last JWT_EXPIRES_IN (7 days by default).
- **Ids** are returned as id (never _id). A malformed id is 400 or 422; an unknown one is 404.
- **Lists** accept page, limit (max 200), sort, q (search) and exact-match field filters, and return pagination in meta.
- **Geometry** is GeoJSON { type: "Point", coordinates: [lng, lat] }; [0, 0] is rejected.
- **Upstream failure is not an HTTP error:** integration payloads carry { ok: false, reason }.

Endpoints by module

Module	Main routes	Role
Auth	POST /auth/register, /auth/login, /auth/change-password; GET/PATCH /auth/me	public / signed-in
1 Dashboard	GET /dashboard/overview, /trend, /activity	public
2 GIS	GET /gis/layers, /gis/heatmap, /gis/within	public
3 Parks & Assets	GET /parks, /parks/near, /parks/:id/health; CRUD /assets, /assets/stats, POST /assets/:id/maintenance	public read; officer write; admin delete
4 Biodiversity	GET /biodiversity/indices, /compare, /seasonality, /species, /observations; POST /indices/preview, /observations/:id/verify	public read; ecologist curate
5 AI	GET /ai/tasks, /stats, /gallery, /images/:id; POST /ai/analyze, /ai/:id/review	signed-in analyse; officer review
6 Sensors	GET /sensors/live, /:id/readings, /:id/anomalies; POST /sensors/refresh, /:id/readings (device only)	public read; officer refresh
7 Citizen	GET /citizen/reports, /my-reports, /stats; POST /reports, /:id/upvote, /:id/review	signed-in submit; officer review
8 Incidents & Alerts	GET /incidents, /triage, /stats; POST /:id/assign, /:id/resolve, /:id/work-order; /alerts	officer (alerts public to read)
9 Maintenance	GET /maintenance, /calendar, /stats; PATCH /:id/progress	public read; officer write
10 Analytics	GET /analytics/summary, trends, /park-comparison, /export, /reports; POST /reports/generate	public; officer export; ecologist reports
11 Assistant	POST /assistant/ask, /search; GET /suggestions, /history/:sessionId	public
12 Admin	/admin/users, /settings, /audit-log, /stats, /recompute-scores, /reindex-assistant, /reseed	admin
Integrations	GET /integrations/status, /weather, /air-quality, /park-conditions, /gbif/:speciesId, /geocode	public

Try it

```
curl -X POST localhost:5000/api/biodiversity/indices/preview \
  -H "Content-Type: application/json" -d "{\"abundances\":[25,25,25,25]}"
# -> richness 4, shannon 1.3863, evenness 1, simpsonDiversity 0.75
```

21. The Database

MongoDB with 18 collections. A park is one document holding its OSM boundary, area, facilities and cached scores, rather than rows spread across joined tables.

Collection	What it holds
Park	6 parks: GeoJSON centre and OSM boundary polygon, area, facilities, source, cached scores.
Asset	Trees, benches, lamps, paths (with a LineString), water bodies, structures; OSM source id; condition; embedded maintenance history; demo flag.
Species	Catalogue only: gbifKey, IUCN category, isInvasive / isIntroduced (GRIIS), photo and credit. No population count.
Observation	Species, park, count, source (gbif, citizen-report, officer-survey ...), verified. The abundance records every index uses.
Sensor / SensorReading	Registry with source (open-meteo, simulated, device), thresholds and cached value; the time series with anomaly verdicts.
CitizenReport	Public submissions with upvotedBy, linked incident or species; demo flag.
Incident / Alert	Computed priority, upvotes, embedded timeline, assignee; alerts deduplicated by dedupeKey.
WorkOrder	Scheduled tasks linked to an asset and optionally an incident; progress and cost.
AiDetection / AiImage	Wildlife inference results with engine, species, probabilities, top ImageNet classes, evidence and review; stored JPEGs, unique by SHA-256.
EcoReport	Frozen metric snapshot, findings, recommendations; draft, published or archived.
User, AuditLog, Setting	Accounts (bcrypt hash, role); append-only change history; singleton settings (weights, thresholds, switches).
ChatMessage, Counter	Assistant history with citations; atomic reference-code sequences.

Design decisions worth defending

- **Abundance is derived, not stored.** Species carries no count; indices aggregate Observation on every read, so they always match the records.
- **Readings are their own narrow collection,** with a compound index on (sensor, recordedAt) serving both the chart query and the detector's rolling window.
- **Maintenance history and timelines are embedded,** because they are always read with their parent, append-only and bounded.
- **Images are stored once by content hash,** so detections and incidents reference /api/ai/images/:id instead of a third-party URL that can disappear.
- **The GIS module has no collection.** Every layer is a projection of another module's data.

22. Testing

```
npm test                                # backend unit + integration suites (node:test)
npm run typecheck                       # frontend: tsc --noEmit
npm run build                           # frontend production build
npm run eval:vision                     # AI accuracy report on 3 wildlife photos
```

Unit suites (backend/test/unit)

Suite	What it checks
aqi	Interpolation, band edges, truncation between bands, 24 h / 8 h averaging, clamping, maximum operator, score inversion.
biodiversity	Even and single-species communities, a hand-computed Shannon value, Margalef, dominance, empty and invalid input, bounded composite score.
priority	Fire outranks a bench repair, bounded 0-100 score, severity clamping, saturating urgency, overdue flag, capped community signal, queue ordering.
anomaly	Spikes and drops flagged, two-of-three voting, insufficient history, constant series (MAD fallback), zero-IQR windows, configurable thresholds.
vision	ImageNet groups do not overlap, probability vectors sum to one, fire needs network evidence, only fire escalates, internal addresses refused, JPEG/PNG only.
services, serialisation	Open-Meteo local time to UTC, Kolkata diurnal hour, intent routing, report findings quote their metrics; response ids, dates and GeoJSON serialise correctly.

Integration suites (backend/test/integration)

The integration suites seed the real open-data snapshot into their own in-memory MongoDB and drive the API over HTTP. They cover every module - for example: map layers are valid GeoJSON; indices are computed from stored observations; inference comes from the pixels, not the request; a confident fire photo opens an incident whose alert clears on resolution; a citizen report escalates to an incident and a work order; role hierarchy, soft-delete and pagination behaviour; reference codes never collide after a delete; an account upvotes once; virtual and simulated sensors refuse posted readings; registration cannot grant a privileged role and a tampered token gains nothing.

> **Test dependencies**
>
> Live Open-Meteo history is skipped under test, so the suite does not depend on the network. The vision model must already be cached or be downloadable, because the AI tests run real inference.

23. Deployment

The backend needs a long-running Node process - it refreshes sensors on a timer and holds the vision model in memory - so it is not suited to serverless functions. A free-tier setup:

1. Database - MongoDB Atlas

Create a free cluster and a database user, allow network access from the host, and copy the mongodb+srv:// connection string.

2. Backend - Render

New, Blueprint, select the repository. render.yaml defines the web service: root directory backend/, Node 20, build "npm ci --omit=dev && npm run model:download" (so the model is downloaded at build time), start "npm start", health check /api/health, NODE_ENV=production, AUTO_SEED=true, and a generated JWT_SECRET. Enter MONGODB_URI and CORS_ORIGIN when prompted. The first boot seeds the database.

3. Frontend - Vercel or Netlify

Import the repository with the root directory set to frontend, and set NEXT_PUBLIC_API_URL to https://<your-render-service>.onrender.com/api before the first build - it is baked in. For Netlify, netlify.toml at the repository root already sets base = "frontend", the build command and Node 20. Leave NEXT_PUBLIC_SHOW_DEMO_ACCOUNTS unset for a public site.

4. Connect them

Put the frontend URL into the backend's CORS_ORIGIN and redeploy the backend.

Any other Node host

```
cd backend
npm ci --omit=dev
npm run model:download
npm start          # with MONGODB_URI, JWT_SECRET, CORS_ORIGIN, NODE_ENV=production
```

> **Production behaviour to know**
>
> The server refuses to start without a real MONGODB_URI, a strong JWT_SECRET and CORS_ORIGIN. Reseeding is refused. The /api rate limit tightens to 300 requests per minute per IP. Rate limiters and the upstream cache are in memory, so they are per process - several instances would need a shared store such as Redis.

24. Limitations

> **Read this chapter carefully**
>
> An examiner who finds a weakness you did not mention will press it. An examiner you have already told will usually move on - and trust everything else you said more.

Limitation	The honest position
No physical sensors	AQI, temperature and humidity are real observations from gridded models, so nearby parks can share a cell. Noise, soil and water are simulated - which makes the water and soil sub-indices, and half of the air sub-index, simulated. A device sensor would use the same ingestion path.
Vision without an API key	The fallback uses pre-trained ImageNet weights, not trained on park imagery: 3/3 on a small set of wildlife photos; species at ImageNet granularity. With a key, identification depends on an external API and its free-tier quota. Hence officer review of every finding.
GBIF counts are not population counts	They are records, dominated by eBird and iNaturalist activity. Lalbagh holds about nine in ten; lightly recorded parks look artificially even; recent months are incomplete.
Pooled diversity index	Mixes units of survey effort across taxa; the per-taxocene indices are the rigorous comparison.
Operational records are demo data	Accounts, citizen reports, incidents, work orders and asset condition scores are flagged demo: true until the portal is really used.
The assistant is not an LLM	Retrieval is TF-IDF with cosine ranking; generation is templates, by design.
JWT in localStorage	Readable by any successful XSS. A hardened deployment should use httpOnly SameSite cookies with CSRF protection.
Server-side image fetching	Private and link-local addresses are refused and redirects re-checked, but the DNS lookup and the fetch are separate, so a DNS-rebinding race remains possible.
Single-process state	Rate limiters and the upstream cache live in memory.

25. One-Page Cheat Sheet

If you remember only five things

- **Real open data, labelled:** OpenStreetMap parks and assets, GBIF species and records, GRIIS invasive flags, Open-Meteo weather and CAMS air quality. Demo and simulated values carry a badge.
- **AQI is the MAXIMUM sub-index,** computed here from CAMS concentrations with CPCB averaging periods and truncation.
- **The health index ignores missing data** by dividing by the weights actually present; missing is null, never 0.
- **The AI is real inference with measured limits:** AI vision (Gemini by default) names wildlife species; the MobileNetV2 fallback scores 3/3 on the labelled wildlife photos; every finding is reviewed by a park officer.
- **GBIF counts are records, not individuals,** and recording effort differs hugely between parks.

Numbers worth knowing

Figure	Value
Parks / modules	6 Bengaluru parks / 12 modules
Open-data snapshot	587 species, 4,175 observation rows, 37,182 GBIF records (Jan 2023 - Sep 2026)
Seeded	339 assets, 33 sensors (18 Open-Meteo, 15 simulated), 3 AI wildlife detections, 6 reports; demo: 12 accounts, 36 citizen reports, 30 incidents, 36 work orders
Collections / routers / page routes	18 / 18 / 16
Vision	Wildlife recognition: AI vision (Gemini by default, free GEMINI_API_KEY); MobileNetV2 1.0 / 224, ~14 MB, ~30-80 ms per image, as evidence and fallback
AI accuracy	Fallback engine 3 / 3 wildlife photos (a smoke test)
Default weights	air 0.25, water 0.20, tree 0.20, biodiversity 0.20, soil 0.15

Questions you should not be caught out by

- **"Is the data real?"** Mostly, and the interface labels which is which (Chapter 5).
- **"Is the AI real?"** Real inference with Google's pre-trained weights; not trained by me; accuracy measured and stated, with the calibration caveat.
- **"Why MongoDB?"** Nested documents, first-class GeoJSON with 2dsphere indexes, and a schema that evolved during the build. Heavy spatial joins would justify PostGIS.
- **"What would you do next?"** Evaluate both vision engines on a held-out set of labelled park photos built from the review queue; real device sensors on the existing endpoint; structured equal-effort biodiversity surveys; embeddings for retrieval; httpOnly cookies.
- **"Show me it working."** Index Calculator with "100, 1, 1, 1" and "25, 25, 25, 25"; then the pond heron photo on AI Monitoring.

> **The most important advice**
>
> Do not claim more than the project does. Every simulated or demonstration value is labelled in the interface, and the AI page states its own accuracy and which engine produced each result. A student who says "real inference, AI vision names the species, the local fallback names the group and was checked on labelled photos" sounds far more credible than one who claims a working detector and cannot say how it was evaluated.